

Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencias de Datos y Matemáticas

Manual del Desarrollador

SIGAM: Sistema Institucional de Gestión para Casa Monarca

Integrantes del equipo

Carola Vázquez Arjona	A01174562
Iker Álvarez Bandrés	A01282986
Juan Pablo Valdés Cárdenas	A00839013
Andrés Vivanco Treviño	A01723358
Juan Fernando González	A01571586
Carlos Eduardo Ramos Martínez	A01286025

Equipo 4 | Grupo 604

Profesores

Raúl Gómez, Anas Wajid y Alberto F. Martínez

Socio Formador

Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P.

Monterrey, Nuevo León

12 de junio de 2026

Índice

1. Descripción General	1
1.1. Propósito del sistema	1
1.2. Alcance	1
1.3. Casos de uso principales	2
2. Arquitectura del Sistema	3
2.1. Tipo de arquitectura	3
2.2. Componentes principales	3
2.3. Diagrama general	4
3. Stack Tecnológico	5
3.1. Lenguajes	5
3.2. Frameworks	5
3.3. Bibliotecas y extensiones	6
3.4. Infraestructura y herramientas	6
4. Estructura del Proyecto	6
4.1. Organización de carpetas	6
4.2. Descripción de módulos	8
5. Configuración del Entorno	9
5.1. Requisitos	9
5.2. Variables de entorno	9
5.3. Archivos de configuración	10
6. Instalación y Ejecución	11
6.1. Pasos para levantar el sistema	11
6.1.1. Entorno de desarrollo local (Docker)	11
6.1.2. Entorno de producción (hosting compartido)	12
6.2. Comandos principales	12
7. Base de Datos	13
7.1. Modelo de datos	13
7.2. Tablas principales	13
7.3. Relaciones	15

8. API / Interfaces	16
8.1. Convenciones generales	16
8.2. Endpoints de autenticación	16
8.3. Endpoints de usuarios	17
8.4. Endpoints de certificados	18
8.5. Endpoints de documentos	18
8.6. Endpoints de comunicación interna	19
8.7. Endpoints de comunicación externa	20
9. Flujos y Lógica de Negocio	21
9.1. Procesos clave del sistema	21
9.1.1. Ciclo de vida de un colaborador	21
9.1.2. Emisión y verificación de certificados	22
9.1.3. Firma digital con zero-knowledge	22
9.1.4. Reglas de comunicación interna	22
9.1.5. Verificación de comunicaciones externas	22
9.1.6. Recuperación administrativa	23
9.2. Reglas importantes	23
9.3. Casos críticos	23
10. Ejemplos de Uso	24
10.1. Caso típico: publicar un anuncio firmado	24
10.2. Caso típico: validar un adjunto antes de persistirlo	25
10.3. Caso típico: verificar acceso con el middleware RBAC	25
10.4. Integración básica: agregar un nuevo tipo de notificación por correo	26
10.5. Ejecución de funciones clave: generación del certificado del sistema	26
11. Manejo de Errores	27
11.1. Estrategia de errores	27
11.2. Códigos de error utilizados	27
11.3. Mensajes comunes	28
11.4. Registro de errores (logs)	29
12. Pruebas	29
12.1. Tipos de tests	29
12.2. Cómo ejecutarlas	30
12.2.1. Pruebas de endpoints con curl	30

12.2.2. Pruebas de validación de archivos	30
12.2.3. Pruebas del cron de integridad	30
12.2.4. Pruebas de verificación criptográfica	31
12.3. Evidencias de pruebas	31
13.Despliegue	31
13.1. Cómo hacerlo	31
13.1.1. Despliegue en entorno de desarrollo (Docker)	31
13.1.2. Despliegue en producción (HostGator Plan Negocios)	32
13.2. Entornos	32
13.3. Consideraciones	33
14.Seguridad	34
14.1. Autenticación	34
14.2. Manejo de datos sensibles	34
14.2.1. Contraseñas	34
14.2.2. Llaves privadas	34
14.2.3. Contenido de documentos y comunicaciones	35
14.3. Buenas prácticas aplicadas	35
14.4. Recomendaciones para desarrolladores futuros	36
15.Guía de Contribución	36
15.1. Convenciones de código	36
15.1.1. PHP	36
15.1.2. JavaScript	37
15.2. Flujo de trabajo (Git)	37
15.3. Cómo agregar funcionalidades	38
16.Problemas Conocidos	39
16.1. Bugs actuales	39
16.2. Limitaciones	39
16.2.1. Limitaciones del hosting compartido (HostGator Plan Negocios) . . .	39
16.2.2. Limitaciones funcionales conscientes	40
17.FAQ Técnica	40
17.1. Problemas comunes al levantar el proyecto	40
17.2. Errores típicos de desarrollo	41

Anexo: Repositorio del proyecto	42
Referencias	42

Índice de figuras

1. Arquitectura de tres capas de SIGAM, mostrando los componentes principales de cada capa y los protocolos de comunicación entre ellas. 4
2. Diagrama entidad-relación del esquema de base de datos de SIGAM. 13

1. Descripción General

1.1. Propósito del sistema

SIGAM (Sistema Institucional de Gestión para Casa Monarca) es una plataforma web desarrollada para resolver la ausencia de mecanismos seguros de gestión de identidades, control de accesos y protección documental en Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P. El sistema centraliza la administración de colaboradores, la emisión y verificación de certificados digitales, la gestión de documentos firmados, y la comunicación interna y externa de la organización, aplicando esquemas criptográficos estándar de la industria para garantizar autenticidad, integridad y trazabilidad en cada operación.

Este manual está dirigido a desarrolladores que necesiten mantener, extender o auditar el sistema. A diferencia del reporte técnico, que documenta las decisiones de diseño y su justificación, este manual responde a la pregunta operativa de *cómo* está construido el sistema: su arquitectura interna, su estructura de código, los flujos de datos entre componentes y las convenciones que se deben seguir para contribuir sin romper la consistencia del proyecto.

1.2. Alcance

El sistema cubre los siguientes módulos funcionales:

- Gestión de identidades con cuatro niveles jerárquicos y control de acceso por área operativa.
- Infraestructura de clave pública (PKI) con autoridad certificadora interna.
- Gestión documental con firma digital y verificación periódica de integridad.
- Comunicación interna mediante anuncios institucionales y mensajes directos firmados.
- Comunicación externa segura con verificación criptográfica de adjuntos y firma S/MIME.

El sistema no cubre, en su versión actual, los siguientes aspectos, que quedan identificados como mejoras para una versión 2:

- Cifrado de la base de datos a nivel motor (Transparent Data Encryption).
- Análisis de contenido con motor antivirus sobre archivos adjuntos.
- Autenticación de doble factor (2FA/TOTP).

- Carpetas de archivos personales por colaborador.
- Módulo de reportes y estadísticas operativas.

El sistema está diseñado para operar en hosting compartido (HostGator Plan Negocios) o en entorno local mediante contenedores Docker, sin requerir infraestructura cloud especializada.

1.3. Casos de uso principales

Los siguientes casos de uso representan los flujos operativos centrales que un desarrollador debe comprender antes de modificar el sistema:

1. **Alta y activación de un colaborador.** El administrador crea un usuario nuevo especificando nombre, correo, nivel y área. El usuario queda inactivo hasta que el administrador lo activa, momento en el cual recibe sus credenciales (o, si es un externo exclusivo de comunicación, su link personal) por correo firmado con S/MIME.
2. **Emisión de certificado digital.** El administrador emite un certificado X.509 para un colaborador. El sistema genera el par de llaves, firma el certificado con la CA interna, y entrega la llave privada al usuario mediante un link de descarga temporal.
3. **Firma digital de contenido.** Un usuario firma un documento, anuncio, mensaje o comunicación externa subiendo su llave privada al formulario correspondiente. La firma se genera localmente en el navegador y se verifica en el backend contra el certificado activo del usuario.
4. **Comunicación entre colaboradores.** Un usuario publica un anuncio o inicia un mensaje directo, sujeto a las restricciones de nivel y área codificadas en el middleware de mensajería.
5. **Recepción de comunicación externa.** Un colaborador externo, sin acceder a la plataforma principal, envía una comunicación firmada desde una página pública dedicada. El sistema verifica la firma y notifica al administrador.
6. **Recuperación administrativa.** Si el administrador principal pierde acceso, la cuenta espejo inicia el procedimiento formal de recuperación, que queda registrado en bitácora y notificado a ambas partes.

2. Arquitectura del Sistema

2.1. Tipo de arquitectura

SIGAM implementa una arquitectura de tres capas (3-tier) con separación estricta de responsabilidades entre presentación, lógica de aplicación y persistencia de datos. Esta elección, frente a alternativas como arquitecturas de microservicios o monolitos modulares más complejos, responde a la necesidad de mantener el sistema simple de operar y desplegar en hosting compartido, sin sacrificar la separación de responsabilidades que facilita el mantenimiento futuro.

Las tres capas se comunican exclusivamente a través de interfaces bien definidas: el frontend nunca accede directamente a la base de datos, y el backend nunca renderiza HTML más allá de las plantillas de correo electrónico. Esta disciplina arquitectónica simplifica las pruebas, permite escalar cada capa de forma independiente si fuera necesario, y reduce el acoplamiento entre componentes.

2.2. Componentes principales

- **Capa de presentación:** aplicación de página única (`frontend.html`) construida en HTML, CSS y JavaScript vanilla, complementada con cinco páginas públicas dedicadas (`verificar.html`, `externo.html`, `reset-password.html`, `terminos.html`, `seed.php`) que atienden flujos sin autenticación previa.
- **Capa de aplicación:** backend PHP 8.2 sobre Apache que expone una API REST mediante un router único (`api/index.php`). Incluye once controladores especializados por recurso, tres módulos de middleware (RBAC, mensajería, validación de archivos), un componente de correo con soporte S/MIME, y dos scripts de tareas programadas.
- **Capa de datos:** base de datos relacional MySQL 8 accedida exclusivamente mediante PDO con prepared statements. El esquema modela usuarios, certificados, documentos, carpetas, comunicaciones internas y externas, correos institucionales, bitácora y tablas auxiliares de configuración.
- **Infraestructura de contenedores:** en entorno de desarrollo, el sistema corre como tres contenedores Docker independientes orquestados mediante `docker-compose.yml`: PHP-Apache, MySQL y phpMyAdmin para administración visual de la base de datos.

2.3. Diagrama general

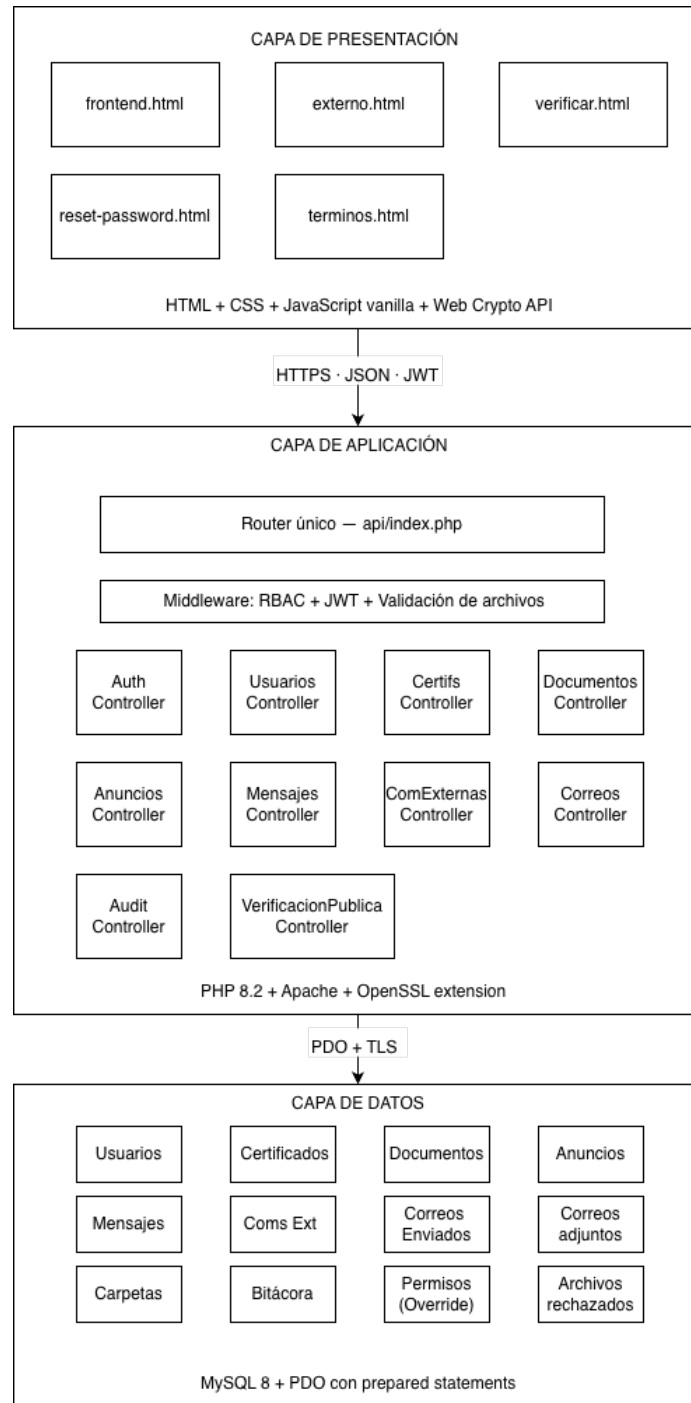


Figura 1: Arquitectura de tres capas de SIGAM, mostrando los componentes principales de cada capa y los protocolos de comunicación entre ellas.

La Figura 1 ilustra el flujo de comunicación entre capas. El frontend se comunica con el backend mediante peticiones HTTPS con payloads JSON, autenticadas con tokens JWT. El

backend se comunica con la base de datos mediante PDO sobre conexiones TLS en producción. Cada controlador del backend invoca el middleware RBAC antes de ejecutar cualquier operación que modifique datos, y registra la operación en la bitácora institucional como parte de la misma transacción.

3. Stack Tecnológico

3.1. Lenguajes

- **PHP 8.2** — lenguaje del backend. Se eligió por su compatibilidad nativa con hosting compartido económico y la madurez de su extensión OpenSSL, que expone todas las primitivas criptográficas requeridas sin bibliotecas externas.
- **JavaScript (ES6+)** — lenguaje del frontend, ejecutado sin transpilación ni bundler. Se utiliza tanto para la lógica de la SPA como para las operaciones criptográficas en el navegador mediante la Web Crypto API.
- **SQL** — lenguaje de consulta para MySQL 8, utilizado exclusivamente a través de prepared statements parametrizados.
- **HTML5 y CSS3** — markup y estilos del frontend, sin frameworks de CSS.

3.2. Frameworks

El proyecto deliberadamente **no utiliza frameworks** de aplicación en ninguna de las dos capas de código:

- En backend, no se usa Laravel, Symfony ni ningún microframework PHP. El router se implementa manualmente en `api/index.php`, y los controladores siguen un patrón de clases simples sin inyección de dependencias automática.
- En frontend, no se usa React, Vue ni Angular. La SPA se construye con funciones JavaScript que manipulan el DOM directamente mediante `innerHTML` y manejadores de eventos.

Esta decisión reduce la curva de aprendizaje para futuros mantenedores del sistema, evita la necesidad de un proceso de build, y elimina dependencias que requerirían actualizaciones de seguridad independientes del propio código del proyecto.

3.3. Bibliotecas y extensiones

- **OpenSSL (extensión nativa de PHP)** — utilizada para generación de pares de llaves RSA, emisión y firma de certificados X.509, verificación de firmas digitales y aplicación de firmas S/MIME sobre correos electrónicos.
- **PDO (PHP Data Objects)** — capa de abstracción para acceso a MySQL con soporte nativo de prepared statements.
- **ZipArchive (extensión nativa de PHP)** — utilizada para inspeccionar la estructura interna de archivos Office (DOCX, XLSX, PPTX) y detectar macros u objetos OLE embebidos.
- **Web Crypto API (nativa del navegador)** — utilizada en el frontend para importar llaves privadas en formato PKCS#8 y generar firmas digitales RSASSA-PKCS1-v1_5 con SHA-256 sin que la llave abandone el navegador.
- **Firestore JWT (o implementación equivalente en PHP puro)** — utilizada para la emisión y verificación de tokens JWT firmados con HS256.

3.4. Infraestructura y herramientas

- **Docker y Docker Compose** — orquestación de contenedores para entorno de desarrollo local.
- **Apache HTTP Server** — servidor web que ejecuta PHP mediante mod_php.
- **MySQL 8** — motor de base de datos relacional.
- **phpMyAdmin** — interfaz web de administración de la base de datos durante desarrollo.
- **Mailtrap** — servidor SMTP de pruebas utilizado durante desarrollo para inspeccionar correos salientes sin enviarlos a destinatarios reales.
- **Git y GitHub** — control de versiones y repositorio remoto del proyecto.

4. Estructura del Proyecto

4.1. Organización de carpetas

La siguiente estructura representa la organización del repositorio:

```

MA2006B/
docker-compose.yml
.env.example
LICENSE
README.md
docs/
    reportes/
    manual_usuario.pdf
    manual_desarrollador.pdf
src/
    api/
        index.php                # Router principal de la API
    config/
        database.php             # Conexión PDO singleton
        env.php                  # Cargador manual de .env
        mail.php                 # Mailer con firma S/MIME
        templates.php            # Plantillas HTML de correos
    controllers/
        AuthController.php
        UsuariosController.php
        CertificadosController.php
        DocumentosController.php
        CarpetasController.php
        AnunciosController.php
        MensajesController.php
        ComunicacionesExternasController.php
        CorreosController.php
        VerificacionPublicaController.php
        AuditController.php
    middleware/
        rbac.php                 # Control de acceso por niveles
        mensajeria.php           # Reglas de comunicación interna
        validacion_archivos.php  # Validación de adjuntos en tres capas
    setup/
        generar_cert_sistema.php
    cron/

```

```

    avisos_expiracion.php
    verificar_hashes_adjuntos.php
frontend.html                # SPA principal
verificar.html               # Verificador público de firmas
externo.html                 # Canal de comunicación externa
reset-password.html
terminos.html
seed.php                     # Inicialización del sistema

```

4.2. Descripción de módulos

api/ — contiene el router único del backend. Todo request HTTP a `/api/{recurso}/{accion}` se procesa aquí, extrayendo el recurso y la acción de la URL e instanciando el controlador correspondiente.

config/ — contiene la configuración compartida del sistema: conexión a base de datos, carga de variables de entorno, lógica de envío de correos con firma S/MIME, y las plantillas HTML reutilizadas por los distintos tipos de notificación.

controllers/ — contiene la lógica de negocio organizada por recurso. Cada controlador expone métodos públicos que corresponden a una acción de la API (crear, listar, ver, actualizar, etc.) y delega la autorización al middleware RBAC antes de ejecutar cualquier operación.

middleware/ — contiene lógica transversal reutilizada por múltiples controladores: verificación de permisos (`rbac.php`), reglas de quién puede comunicarse con quién (`mensajeria.php`), y validación de archivos adjuntos en tres capas (`validacion_archivos.php`).

setup/ — contiene scripts ejecutables una sola vez durante la instalación inicial del sistema, como la generación del certificado dedicado para firma S/MIME.

cron/ — contiene scripts diseñados para ejecutarse periódicamente mediante `crontab`, fuera del flujo de peticiones HTTP: avisos de expiración de certificados y verificación de integridad de adjuntos.

Archivos HTML en la raíz de src/ — corresponden a las páginas servidas directamente por Apache. `frontend.html` es la SPA principal que requiere autenticación; las demás son páginas públicas que atienden flujos específicos sin necesidad de login.

5. Configuración del Entorno

5.1. Requisitos

- **Docker** versión 24 o superior y **Docker Compose** versión 2 o superior, para entorno de desarrollo local.
- **PHP 8.2** con las extensiones `openssl`, `pdo_mysql`, `zip` y `mbstring` habilitadas. Estas extensiones son requisito indispensable: `openssl` para toda la capa criptográfica, `zip` para la inspección de archivos Office, y `pdo_mysql` para la conexión a base de datos.
- **MySQL 8** o superior, con soporte para el motor de almacenamiento InnoDB (requerido para las llaves foráneas del esquema).
- **Apache 2.4** o superior, con `mod_php` habilitado.
- **Navegador moderno** con soporte para Web Crypto API: Chrome 90+, Firefox 90+, Safari 14+, Edge 90+.
- **Git** para clonar el repositorio y gestionar control de versiones.
- Para producción: hosting compartido con soporte para PHP 8.2, MySQL 8, extensiones OpenSSL y ZipArchive habilitadas. El proyecto fue validado contra HostGator Plan Negocios.

5.2. Variables de entorno

El sistema utiliza un archivo `.env` en la raíz del proyecto para separar la configuración del código fuente. El cargador manual implementado en `config/env.php` lee este archivo y expone sus valores mediante la función `env($clave, $default)`.

Tabla 1: Variables de entorno requeridas por el sistema.

Variable	Descripción
APP_URL	URL base del sistema, utilizada para construir links en correos electrónicos (ej. <code>http://localhost:8080</code> en desarrollo, dominio real en producción)

Variable	Descripción
DB_HOST	Hostname del servidor MySQL (<code>mysql</code> en Docker, o el host correspondiente en producción)
DB_NAME	Nombre de la base de datos (<code>casa_monarca</code>)
DB_USER	Usuario de conexión a MySQL
DB_PASS	Contraseña de conexión a MySQL
JWT_SECRET	Cadena secreta utilizada para firmar y verificar tokens JWT con HS256. Debe ser una cadena aleatoria de al menos 64 caracteres en producción
MAIL_HOST	Hostname del servidor SMTP (<code>sandbox.smtp.mailtrap.io</code> en desarrollo)
MAIL_PORT	Puerto del servidor SMTP (2525 para Mailtrap)
MAIL_USERNAME	Usuario de autenticación SMTP
MAIL_PASSWORD	Contraseña de autenticación SMTP
MAIL_FROM_EMAIL	Dirección de correo remitente institucional
MAIL_FROM_NAME	Nombre del remitente mostrado en los correos (Casa Monarca)

5.3. Archivos de configuración

- `.env` — variables de entorno descritas en la tabla anterior. Este archivo **nunca** debe subirse al control de versiones; el repositorio incluye un `.env.example` con la estructura esperada pero sin valores reales.
- `docker-compose.yml` — define los tres servicios del entorno de desarrollo (PHP-Apache, MySQL, phpMyAdmin), sus volúmenes, puertos expuestos y variables de entorno propagadas a cada contenedor.
- `config/database.php` — define la función `getDB()` que instancia y reutiliza una conexión PDO singleton durante toda la ejecución de una petición.
- **Certificado del sistema** — ubicado en `/var/certificados/sistema/` dentro del contenedor (o ruta equivalente en producción), contiene el par de llaves dedicado para

firma S/MIME de correos salientes. Se genera una sola vez mediante el script de `setup/` y requiere permisos restringidos de lectura para el usuario que ejecuta PHP (`www-data`).

6. Instalación y Ejecución

6.1. Pasos para levantar el sistema

6.1.1. Entorno de desarrollo local (Docker)

1. Clonar el repositorio:

```
git clone https://github.com/carolavazquez/MA2006B.git
cd MA2006B
```

2. Crear el archivo `.env` a partir del ejemplo provisto:

```
cp .env.example .env
```

Editar los valores según el entorno (credenciales de Mailtrap, secretos de JWT, etc.).

3. Levantar los contenedores:

```
docker compose up -d
```

4. Verificar que los tres contenedores están corriendo:

```
docker compose ps
```

Deben aparecer los servicios `php`, `mysql` y `phpmyadmin` con estado `Up`.

5. Generar el certificado dedicado del sistema para firma S/MIME:

```
docker compose exec php php /var/www/html/setup/generar_cert_sistema.php
```

6. Inicializar la base de datos y crear el administrador inicial:

```
curl http://localhost:8080/seed.php
```

Este script devuelve las credenciales temporales del administrador y el link de descarga de su certificado. Es indispensable guardar ambos de inmediato, ya que el link expira en 24 horas y la contraseña no se vuelve a mostrar.

7. Acceder al sistema en `http://localhost:8080/frontend.html`.

6.1.2. Entorno de producción (hosting compartido)

1. Subir el contenido de `src/` a la raíz pública del hosting mediante FTP o el administrador de archivos de cPanel.
2. Crear la base de datos MySQL desde el panel del proveedor e importar el esquema inicial.
3. Crear y configurar el archivo `.env` en el servidor con las credenciales de producción, incluyendo el dominio real en `APP_URL` y las credenciales del servidor SMTP institucional.
4. Ejecutar el script de generación del certificado del sistema desde una terminal SSH si el proveedor lo permite, o mediante un script PHP invocado una sola vez desde el navegador.
5. Ejecutar `seed.php` una única vez para inicializar el administrador.
6. Configurar el cron del proveedor para ejecutar diariamente `cron/avisos_expiracion.php` y `cron/verificar_hashes_adjuntos.php`.

6.2. Comandos principales

Tabla 2: Comandos de uso frecuente durante el desarrollo.

Comando	Propósito
<code>docker compose up -d</code>	Levanta los contenedores en segundo plano
<code>docker compose down</code>	Detiene y elimina los contenedores
<code>docker compose ps</code>	Muestra el estado de los contenedores
<code>docker compose logs php -tail 50</code>	Muestra los últimos 50 registros de log del contenedor PHP
<code>docker compose exec php bash</code>	Abre una shell interactiva dentro del contenedor PHP
<code>docker compose exec php php archivo.php</code>	Ejecuta un script PHP puntual dentro del contenedor

Comando	Propósito
<pre>docker compose exec -it db mysql -u root -p casa_monarca</pre>	Abre un cliente MySQL interactivo contra la base de datos

7. Base de Datos

7.1. Modelo de datos

El esquema de SIGAM está diseñado bajo un modelo relacional normalizado, con identificadores UUID v4 como llave primaria en todas las entidades. Esta decisión evita exponer secuencias predecibles (como los autoincrementales tradicionales) que pudieran facilitar enumeración de registros por parte de un atacante. Todas las relaciones de llave foránea utilizan el tipo VARCHAR(36) para almacenar los UUID en su representación de texto estándar.

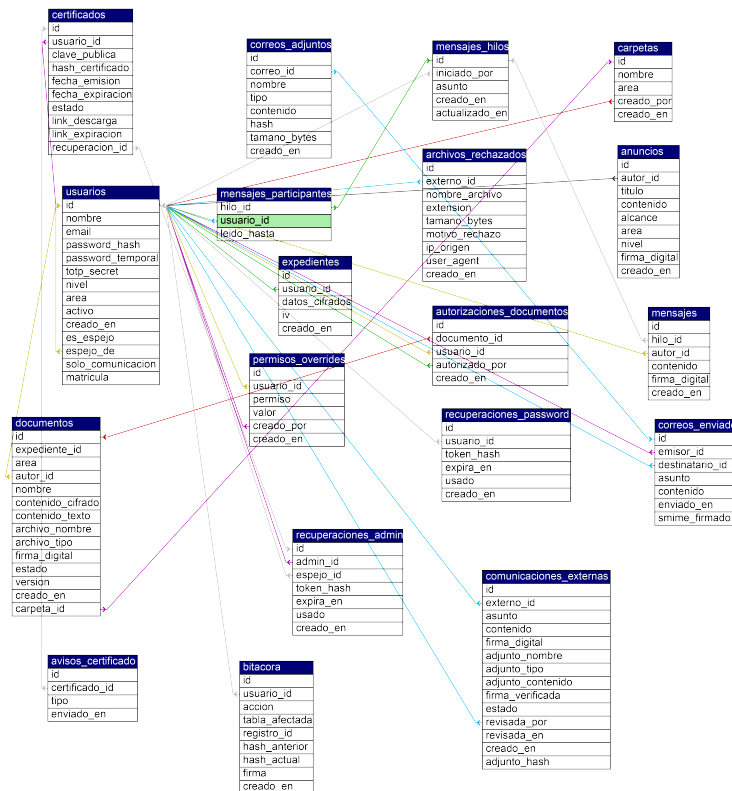


Figura 2: Diagrama entidad-relación del esquema de base de datos de SIGAM.

7.2. Tablas principales

Tabla 3: Descripción de las tablas principales del esquema.

Tabla	Descripción
usuarios	Almacena colaboradores y externos. Incluye nivel jerárquico, área operativa, matrícula institucional, contraseña hasheada con bcrypt, y flags para cuenta espejo y externos exclusivos de comunicación.
certificados	Almacena certificados X.509 emitidos por la CA interna del sistema, incluyendo llave pública, estado (activo/revocado) y vigencia.
carpetas	Organiza el repositorio documental por área operativa.
documentos	Almacena documentos institucionales con su contenido codificado, hash de integridad y firma digital opcional del autor.
anuncios	Comunicados institucionales de tipo broadcast con firma digital obligatoria.
mensajeshilos	Agrupar conversaciones directas entre dos o más usuarios.
mensajes participantes	Relación N a M entre hilos y usuarios participantes.
mensajes	Mensajes individuales dentro de un hilo, con firma digital opcional.
comunicaciones externas	Comunicaciones firmadas recibidas de colaboradores externos, incluyendo adjunto, su hash y el estado de verificación de la firma.
correos enviados	Histórico de correos institucionales enviados por administradores o coordinadores hacia externos.
correos adjuntos	Adjuntos múltiples asociados a un correo institucional enviado.
archivos rechazados	Cuarentena de intentos de envío de adjuntos que fallaron la validación, con metadatos forenses.

Tabla	Descripción
<code>permisos overrides</code>	Excepciones individuales de permisos sobre el modelo de roles base.
<code>recuperaciones admin</code>	Registro de procedimientos de recuperación administrativa iniciados por la cuenta espejo.
<code>bitacora</code>	Registro inmutable de operaciones sensibles del sistema.
<code>avisos certificado</code>	Control de avisos de expiración enviados por certificado, para evitar duplicados del cron diario.

7.3. Relaciones

Las relaciones principales del esquema son las siguientes:

- `usuarios 1:N certificados` — un usuario puede tener múltiples certificados a lo largo del tiempo (histórico de emisión, renovación y revocación).
- `usuarios 1:N documentos` — como autor del documento.
- `carpetas 1:N documentos` — relación opcional, ya que un documento puede no estar clasificado en ninguna carpeta.
- `usuarios 1:N anuncios` — como autor del anuncio.
- `usuarios 1:N mensajes_hilos` — como iniciador del hilo.
- `mensajes_hilos 1:N mensajes` — con eliminación en cascada.
- `mensajes_hilos N:M usuarios` — a través de la tabla intermedia `mensajes_participantes`, también con eliminación en cascada.
- `usuarios 1:N comunicaciones_externas` — como externo emisor.
- `usuarios 1:N correos_enviados` — en dos roles distintos: como emisor y como destinatario, mediante llaves foráneas separadas.
- `correos_enviados 1:N correos_adjuntos` — con eliminación en cascada.
- `usuarios 1:N recuperaciones_admin` — también en dos roles: quien inicia el procedimiento (cuenta espejo) y el administrador objeto de la recuperación.

- **usuarios 1:N bitacora** — toda operación queda asociada al usuario que la ejecutó, salvo casos de eventos automatizados del sistema donde el campo puede ser nulo.
- **certificados 1:N avisos_certificado** — control de los avisos de expiración enviados.

La gran mayoría de las llaves foráneas hacia **usuarios** no tienen eliminación en cascada, ya que la baja de un usuario no debe eliminar su histórico de operaciones, documentos firmados o comunicaciones. En su lugar, el sistema utiliza el flag **activo** para desactivar usuarios sin perder la trazabilidad asociada a su actividad pasada.

8. API / Interfaces

8.1. Convenciones generales

Todos los endpoints siguen el patrón `/api/{recurso}/{accion}`, despachados por el router único en `api/index.php`. Las peticiones y respuestas usan formato JSON, salvo los endpoints que reciben archivos multipart (creación de documentos, envío de comunicaciones externas con adjuntos, correos institucionales con adjuntos). La autenticación se realiza mediante el header `Authorization: Bearer {token}`, donde `{token}` es un JWT obtenido al hacer login. Los endpoints públicos, que no requieren este header, están marcados explícitamente en cada caso.

Las respuestas exitosas incluyen la clave `"status": .ok` junto con los datos solicitados. Las respuestas de error incluyen la clave `.error` con un mensaje descriptivo y el código HTTP correspondiente (400 para solicitudes inválidas, 401 para autenticación fallida, 403 para autorización insuficiente, 404 para recursos no encontrados, 500 para errores del servidor).

8.2. Endpoints de autenticación

Tabla 4: Endpoints del recurso de autenticación.

Endpoint	Método	Descripción
<code>/api/auth/login</code>	POST	Valida credenciales y emite un token JWT. Público.

Endpoint	Método	Descripción
/api/auth/me	GET	Devuelve los datos del usuario autenticado a partir del token.
/api/auth/verificar-certificado	POST	Valida que un certificado subido corresponda al usuario autenticado.

Ejemplo de uso — login:

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "admin@casamonarca.org",
  "password": "contrase a_temporal"
}
```

```
# Respuesta exitosa
{
  "status": "ok",
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "usuario": { "id": "...", "nombre": "...", "nivel": 1 }
}
```

8.3. Endpoints de usuarios

Endpoint	Método	Descripción
/api/usuarios/crear	POST	Crea un nuevo colaborador. Requiere nivel 1, permiso w.
/api/usuarios/activar	POST	Activa una cuenta y dispara el correo de credenciales o link de comunicación. Requiere nivel 1, permiso x.
/api/usuarios/listar	GET	Lista colaboradores según el alcance del usuario autenticado.
/api/usuarios/obtener-permisos	GET	Devuelve los permisos efectivos de un usuario específico.

/api/usuarios/actualizar-permisos	POST	Modifica los overrides individuales de un usuario. Requiere nivel 1, permiso e .
/api/usuarios/solicitar-reset	POST	Inicia el flujo de restablecimiento de contraseña. Público.

8.4. Endpoints de certificados

Tabla 6: Endpoints del recurso de certificados.

Endpoint	Método	Descripción
/api/certificados/generar	POST	Emite un certificado nuevo para un usuario. Requiere nivel 1.
/api/certificados/renovarmio	POST	Renueva el certificado propio del usuario autenticado.
/api/certificados/reemitirmio	POST	Reemite el certificado vinculado a una recuperación administrativa reciente.
/api/certificados/revocar	POST	Revoca un certificado activo. Requiere nivel 1.
/api/certificados/verificar	POST	Verifica una firma contra el certificado activo de un usuario.

8.5. Endpoints de documentos

Tabla 7: Endpoints del recurso de documentos.

Endpoint	Método	Descripción
/api/documentos/crear	POST	Crea un documento con archivo adjunto en multipart. Requiere permiso w sobre el área.
/api/documentos/listar	GET	Lista documentos, filtrable por carpeta_id .

Endpoint	Método	Descripción
/api/documentos/ver	GET	Devuelve el detalle completo de un documento, incluyendo su contenido codificado.
/api/documentos/firmar	POST	Aplica la firma digital de un usuario sobre un documento existente.
/api/documentos/mover	POST	Reasigna un documento a otra carpeta.

Ejemplo de uso — firmar documento:

```
POST /api/documentos/firmar
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{
  "documento_id": "fd400e6e-665a-406a-86b2-6ba2a469d599",
  "firma": "MIIBIjANBgkqhkiG9w0BAQEFAA0CAQ8A..."
}
```

8.6. Endpoints de comunicación interna

Tabla 8: Endpoints de anuncios y mensajes directos.

Endpoint	Método	Descripción
/api/anuncios/crear	POST	Publica un anuncio. Requiere firma digital obligatoria.
/api/anuncios/listar	GET	Lista anuncios visibles para el usuario autenticado.
/api/mensajes/contactos	GET	Lista los usuarios con quienes el autenticado puede iniciar conversación.
/api/mensajes/iniciar	POST	Crea un hilo nuevo con el primer mensaje. Firma opcional.

Endpoint	Método	Descripción
/api/mensajes/responder	POST	Agrega un mensaje a un hilo existente. Firma opcional.
/api/mensajes/listar-hilos	GET	Lista los hilos del usuario autenticado.
/api/mensajes/ver	GET	Devuelve el detalle completo de un hilo con sus mensajes.

8.7. Endpoints de comunicación externa

Tabla 9: Endpoints de comunicaciones externas, correos institucionales y verificación pública.

Endpoint	Método	Descripción
/api/externos/info	GET	Devuelve los datos básicos de un externo a partir de su ID. Público.
/api/externos/enviar	POST	Recibe una comunicación firmada de un externo, con adjunto opcional en multipart. Público.
/api/externos/listar	GET	Lista comunicaciones externas recibidas. Requiere nivel 1.
/api/externos/ver	GET	Devuelve el detalle de una comunicación externa.
/api/externos/actualizar-estado	POST	Cambia el estado de revisión de una comunicación.
/api/correos/destinatarios	GET	Lista externos disponibles para enviar correo institucional.
/api/correos/enviar	POST	Envía un correo institucional firmado con S/MIME, con adjuntos múltiples en multipart.
/api/correos/listar	GET	Lista correos institucionales enviados.

Endpoint	Método	Descripción
/api/correos/adjuntos	GET	Lista los adjuntos de un correo institucional.
/api/correos/descargar-adjunto	GET	Devuelve el contenido de un adjunto específico.
/api/verificacion/smime	POST	Verifica la firma S/MIME de un archivo .eml subido. Público.

Ejemplo de uso — verificación pública de correo:

```
POST /api/verificacion/smime
Content-Type: multipart/form-data
```

```
eml: [archivo .eml adjunto]
```

```
# Respuesta
{
  "status": "valido",
  "mensaje": "Este correo fue firmado por el sistema de Casa Monarca...",
  "detalles": {
    "remitente": "Casa Monarca <no-reply@casamonarca.org>",
    "asunto": "Tu certificado digital ha sido emitido",
    "fecha": "Tue, 17 Jun 2026 10:32:00 +0000"
  }
}
```

9. Flujos y Lógica de Negocio

9.1. Procesos clave del sistema

9.1.1. Ciclo de vida de un colaborador

Un usuario nuevo pasa por tres estados: creado (inactivo, sin credenciales enviadas), activo (con acceso operativo) y eventualmente inactivo (baja sin eliminación del histórico). La creación genera una contraseña temporal que se almacena en `password_temporal` hasta el momento de la activación, donde se hashea y se limpia ese campo. Esta separación entre creación y activación permite al administrador preparar varios usuarios sin disparar notificaciones masivas, activándolos de manera escalonada cuando corresponda.

9.1.2. Emisión y verificación de certificados

La emisión genera un par de llaves RSA-2048, construye un CSR con los datos del titular y lo firma con la CA interna del sistema. La llave privada se coloca en un directorio de descarga con un link único de expiración de 24 horas; transcurrido ese plazo o tras la descarga, el archivo se elimina del servidor. La verificación posterior de cualquier firma generada por ese usuario consulta el certificado **activo en el momento de la firma**, no necesariamente el más reciente, lo que permite verificar firmas históricas incluso después de que el certificado haya expirado o sido renovado.

9.1.3. Firma digital con zero-knowledge

Este es el flujo criptográfico central del sistema. El usuario nunca sube su llave privada al servidor: la importa localmente en su navegador, firma el contenido y solo transmite la firma resultante. El backend reconstruye exactamente la misma cadena que firmó el cliente y verifica matemáticamente que la firma corresponda a esa cadena y al certificado activo del usuario. Cualquier discrepancia, por mínima que sea, invalida la verificación.

9.1.4. Reglas de comunicación interna

La función `puedeMandarMensajeA($emisor, $receptor)` codifica la matriz completa de quién puede escribirle a quién: el administrador puede comunicarse con cualquier colaborador; los coordinadores pueden escribir al administrador, a otros coordinadores de cualquier área y a colaboradores de su propia área; los niveles operativos y de consulta están restringidos a su propia área y al administrador. Esta función se consulta tanto al construir la lista de contactos disponibles en el frontend como al validar el envío real en el backend, evitando que un cliente manipulado pueda saltarse la restricción.

9.1.5. Verificación de comunicaciones externas

Cuando un externo envía una comunicación, el backend reconstruye la cadena firmable exacta (`asunto|contenido|base64(adjunto)`) y la verifica contra el certificado activo de ese externo específico. Si el adjunto incluye un archivo, este pasa primero por las tres capas de validación de formato antes de calcular la cadena firmable, garantizando que únicamente archivos válidos lleguen a la etapa de verificación criptográfica.

9.1.6. Recuperación administrativa

El flujo de recuperación involucra dos cuentas distintas: la cuenta espejo, que inicia el procedimiento, y el administrador principal, objeto de la recuperación. Al iniciarse, se crea un registro en `recuperaciones_admin` y se notifican ambas partes por correo firmado. La cuenta espejo puede entonces resetear la contraseña del administrador y emitir un certificado nuevo, el cual queda enlazado al registro de recuperación mediante el campo `recuperacion_id` para trazabilidad.

9.2. Reglas importantes

- La firma es **obligatoria** para anuncios y **opcional** para mensajes directos y documentos.
- Un usuario solo puede tener un certificado **activo** a la vez; emitir uno nuevo no revoca automáticamente el anterior, pero la verificación siempre prioriza el certificado vigente en el momento de la firma consultada.
- Los permisos efectivos de un usuario se calculan combinando los permisos base de su nivel con los **overrides** individuales; un override puede tanto otorgar como revocar un permiso específico.
- Las comunicaciones externas requieren que el externo tenga un certificado **activo**; si su certificado expiró, el envío se rechaza incluso si la firma es matemáticamente correcta contra un certificado vencido.
- Los adjuntos rechazados nunca se persisten en la tabla principal; únicamente se registran sus metadatos en la tabla de cuarentena.

9.3. Casos críticos

- **Pérdida simultánea de acceso del administrador y la cuenta espejo:** requiere intervención técnica directa sobre la base de datos, ya que el sistema deliberadamente no incluye una puerta trasera adicional para evitar superficies de ataque innecesarias.
- **Certificado del administrador expirado sin recuperación previa:** el administrador no puede emitir certificados ni ejecutar ciertas operaciones que requieren su propia firma hasta renovar su certificado, lo cual requiere que tenga acceso de login vigente (contraseña) incluso sin certificado activo.

- **Archivo Office con macros legítimas:** el sistema rechaza categóricamente cualquier archivo Office con `vbaProject.bin` detectado, sin excepción. Si un colaborador necesita compartir un archivo con macros legítimas, debe coordinarse fuera del sistema o guardar el archivo sin macros antes de subirlo.
- **Modificación directa de la base de datos fuera del flujo de la aplicación:** cualquier alteración manual de un adjunto sin pasar por los controladores no actualiza su hash de integridad, lo que será detectado por el cron diario de verificación y generará una alerta, pero no de forma inmediata sino hasta la siguiente ejecución programada.

10. Ejemplos de Uso

10.1. Caso típico: publicar un anuncio firmado

El siguiente ejemplo ilustra el flujo completo desde la perspectiva de un desarrollador que necesita reproducir o depurar este caso de uso.

Paso 1 — Firma en el navegador (frontend):

```
const pemText = await keyFile.text();
const privateKey = await importarLlavePEM(pemText);

const contenidoFirmable = titulo + '|' + contenido;
const bytes = new TextEncoder().encode(contenidoFirmable);
const firmaBuffer = await window.crypto.subtle.sign(
  { name: 'RSASSA-PKCS1-v1_5' },
  privateKey,
  bytes
);
const firma = btoa(String.fromCharCode(...new Uint8Array(firmaBuffer)));
```

Paso 2 — Envío al backend:

```
const r = await API.post('/api/anuncios/crear', {
  titulo, contenido, alcance, firma
});
```

Paso 3 — Verificación en backend (PHP):

```
$contenido_firmable = $titulo . '|' . $contenido;
$firma_bin = base64_decode($firma);
$clave_publica = openssl_get_publickey($cert['clave_publica']);
$verificado = openssl_verify(
  $contenido_firmable,
```

```

        $firma_bin,
        $clave_publica,
        OPENSSL_ALGO_SHA256
    );

    if ($verificado !== 1) {
        return $this->error(401, 'La firma digital no es v lida');
    }

```

10.2. Caso típico: validar un adjunto antes de persistirlo

```

require_once __DIR__ . '/../middleware/validacion_archivos.php';

$validacion = ValidacionArchivos::validar($_FILES['adjunto']);

if (!$validacion['ok']) {
    ValidacionArchivos::registrarRechazo(
        $usuario_id,
        $_FILES['adjunto'],
        $validacion['error']
    );
    return $this->error(400, $validacion['error']);
}

// $validacion contiene: extension, mime, tama o , hash
$adjunto_hash = $validacion['hash'];

```

10.3. Caso típico: verificar acceso con el middleware RBAC

Cualquier método de controlador que requiera autorización debe invocar `verificarAcceso` como primera línea:

```

public function crear() {
    // nivel m nimo 2, sin rea espec fica, requiere permiso de
    // escritura
    $payload = verificarAcceso(2, null, 'w');

    // $payload contiene: sub (id del usuario), nivel, area
    $usuario_id = $payload['sub'];

    // ... resto de la l gica del m todo
}

```

10.4. Integración básica: agregar un nuevo tipo de notificación por correo

Para agregar una plantilla de correo nueva, se sigue el patrón existente en `config/templates.php`:

```
public static function miNuevaPlantilla($nombre, $detalle) {
    $appUrl = env('APP_URL', 'http://localhost:8080');
    $cuerpo = '
        <h2 style="margin-top:0;color:#c84c2a">T tulo del aviso</h2>
        <p>Hola ' . htmlspecialchars($nombre) . ',</p>
        <p>' . htmlspecialchars($detalle) . '</p>';
    return self::layout('Asunto del correo', $cuerpo);
}
```

Y se invoca desde el controlador correspondiente:

```
Mailer::send(
    $usuario['email'],
    'Asunto del correo',
    Templates::miNuevaPlantilla($usuario['nombre'], $detalle)
);
```

Todo correo enviado mediante `Mailer::send()` se firma automáticamente con S/MIME sin necesidad de configuración adicional por parte de quien agrega la plantilla.

10.5. Ejecución de funciones clave: generación del certificado del sistema

El certificado dedicado para firma S/MIME se genera una sola vez durante la instalación inicial:

```
docker compose exec php php /var/www/html/setup/generar_cert_sistema.php
```

Internamente, este script ejecuta:

```
$par_claves = openssl_pkey_new([
    'digest_alg'      => 'sha256',
    'private_key_bits' => 2048,
    'private_key_type' => OPENSSL_KEYTYPE_RSA
]);

$csr = openssl_csr_new($dn, $par_claves);
$certificado = openssl_csr_sign($csr, null, $par_claves, 3650);

openssl_x509_export($certificado, $crt_string);
```



```
openssl_pkey_export($par_claves, $key_string);
```

El resultado se almacena en `/var/certificados/sistema/` con permisos restringidos para el usuario `www-data`.

11. Manejo de Errores

11.1. Estrategia de errores

El sistema sigue una estrategia consistente de manejo de errores basada en códigos de estado HTTP estándar y respuestas JSON estructuradas. Cada controlador expone un método privado `error($codigo, $mensaje)` que establece el código de respuesta HTTP mediante `http_response_code()` y devuelve un arreglo con la clave `.error` y el mensaje descriptivo correspondiente.

```
private function error($code, $mensaje) {  
    http_response_code($code);  
    return ['error' => $mensaje];  
}
```

Esta estrategia permite que el frontend distinga programáticamente entre tipos de error sin necesidad de parsear mensajes en texto libre, ya que el código HTTP por sí mismo indica la naturaleza del error.

11.2. Códigos de error utilizados

Tabla 10: Códigos HTTP de error utilizados en el sistema y su significado.

Código	Significado	Casos de uso típicos
400	Solicitud inválida	Campos requeridos faltantes, formato de datos incorrecto, área o nivel inválido
401	No autenticado / firma inválida	Token JWT expirado o ausente, verificación de firma digital fallida
403	Acceso denegado	Nivel insuficiente, permiso no habilitado, intento de comunicación fuera del ámbito permitido

Código	Significado	Casos de uso típicos
404	Recurso no encontrado	Usuario, documento, certificado o comunicación inexistente
409	Conflicto	Email duplicado al crear usuario, usuario ya activo
500	Error del servidor	Fallo de transacción en base de datos, error al firmar S/MIME, fallo de conexión SMTP

11.3. Mensajes comunes

Tabla 11: Mensajes de error frecuentes y su causa.

Mensaje	Causa típica
“La firma digital no es válida”	El contenido firmado en el cliente no coincide con el reconstruido en el servidor, o se usó una llave que no corresponde al certificado activo
“Tu cuenta no tiene el permiso ‘w’ habilitado”	El middleware RBAC detectó que el permiso requerido no está en los permisos efectivos del usuario
“No tienes permitido escribirle a este usuario”	La función <code>puedeMandarMensajeA</code> rechazó la combinación de nivel y área entre emisor y receptor
“El contenido del archivo no corresponde a su extensión declarada”	Los magic bytes del archivo no coinciden con el formato esperado para la extensión proporcionada

Mensaje	Causa típica
“El archivo de Office contiene macros u objetos embebidos”	Se detectó <code>vbaProject.bin</code> o un directorio <code>embeddings/</code> dentro del ZIP del documento Office
“Necesitas un certificado activo para firmar”	El usuario no tiene ningún certificado con estado <code>activo</code> y <code>fecha_expiracion</code> vigente

11.4. Registro de errores (logs)

Los errores no controlados de PHP (excepciones no capturadas, errores fatales) se registran automáticamente en el log de Apache, accesible mediante:

```
docker compose logs php --tail 50
```

Adicionalmente, ciertos eventos de fallo silencioso se registran explícitamente con `error_log()` para facilitar diagnóstico sin interrumpir el flujo de la aplicación. Un ejemplo es la falla en la firma S/MIME de un correo, donde el sistema continúa con el envío sin firma en lugar de bloquear la notificación, pero deja constancia del fallo:

```
if (!$signedMessage) {
    error_log('Mailer: no se pudo firmar el correo, enviando sin firma');
    return self::sendRaw(...);
}
```

Las operaciones sensibles exitosas se registran en la tabla `bitacora` como mecanismo de auditoría, complementario al log técnico de errores. Mientras los logs de Apache son volátiles y rotan periódicamente, la bitácora es persistente e inmutable, sirviendo como fuente de verdad para reconstruir el histórico de operaciones del sistema.

12. Pruebas

12.1. Tipos de tests

El sistema fue validado mediante pruebas manuales estructuradas sobre cada flujo crítico, dado el alcance del proyecto académico y los plazos de las dos etapas del reto. No se implementó un framework de pruebas automatizadas (como PHPUnit) en esta versión, lo cual queda identificado como mejora para versiones futuras. Las pruebas realizadas se clasifican en las siguientes categorías:

- **Pruebas funcionales:** verifican que cada endpoint produce el resultado esperado ante entradas válidas, siguiendo los casos de uso documentados en la sección de Resultados del reporte técnico.
- **Pruebas de seguridad dirigidas:** verifican que las validaciones de autorización, firma digital y validación de archivos rechazan correctamente entradas maliciosas o manipuladas.
- **Pruebas de regresión manual:** se ejecutan después de cada cambio significativo para confirmar que flujos previamente funcionales no se rompieron.
- **Pruebas de integración:** verifican que la cadena completa frontend-backend-base de datos-correo funciona de extremo a extremo, incluyendo la verificación de correos en Mailtrap durante desarrollo.

12.2. Cómo ejecutarlas

12.2.1. Pruebas de endpoints con curl

Los endpoints pueden probarse directamente desde la línea de comandos sin necesidad del frontend:

```
curl -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"admin@casamonarca.org","password":"contrase a"}'
```

12.2.2. Pruebas de validación de archivos

Para verificar la detección de archivos renombrados maliciosamente:

```
# Crear un archivo de prueba renombrado
head -c 100 /bin/ls > test.txt

# Subirlo a través de externo.html y confirmar el rechazo
```

12.2.3. Pruebas del cron de integridad

```
docker compose exec php php /var/www/html/cron/verificar_hashes_adjuntos.php
```

Para simular una discrepancia detectable, se modifica manualmente el contenido de un adjunto en base de datos sin actualizar su hash, y se vuelve a ejecutar el cron, confirmando que detecta la inconsistencia y dispara la alerta correspondiente.

12.2.4. Pruebas de verificación criptográfica

Para confirmar que la simetría entre la firma en el navegador y la verificación en el backend funciona correctamente:

```
# Verificar que OpenSSL puede firmar correctamente desde PHP
docker compose exec php php -r "
\$tmpIn = tempnam(sys_get_temp_dir(), 'in');
\$tmpOut = tempnam(sys_get_temp_dir(), 'out');
file_put_contents(\$tmpIn, 'Content-Type: text/plain\r\n\r\nHola');
\$ok = openssl_pkcs7_sign(
    \$tmpIn, \$tmpOut,
    'file:///var/certificados/sistema/sistema.crt',
    'file:///var/certificados/sistema/sistema.key',
    ['Subject' => 'Test'],
    PKCS7_DETACHED
);
echo 'firmado: ' . (\$ok ? 'true' : 'false') . PHP_EOL;
"
```

12.3. Evidencias de pruebas

Las evidencias detalladas de cada caso de uso (capturas de pantalla, respuestas JSON capturadas y registros de bitácora) se encuentran organizadas por módulo en el repositorio del proyecto, referenciadas también en la tabla de casos de uso del reporte técnico.

13. Despliegue

13.1. Cómo hacerlo

El despliegue de SIGAM varía según el entorno destino. A continuación se describen los procedimientos para los dos entornos soportados.

13.1.1. Despliegue en entorno de desarrollo (Docker)

El despliegue local utiliza Docker Compose para orquestar los tres servicios necesarios. Los pasos completos están documentados en la Sección 6 (Instalación y Ejecución). En resumen: clonar repositorio, configurar `.env`, levantar contenedores, generar certificado del sistema, ejecutar `seed.php`.

13.1.2. Despliegue en producción (HostGator Plan Negocios)

1. **Preparación de archivos:** comprimir el contenido de `src/` en un archivo ZIP, excluyendo el `.env` de desarrollo.
2. **Subida vía cPanel:** utilizar el Administrador de Archivos de cPanel para subir y descomprimir el ZIP en el directorio público (`public_html` o subdirectorio correspondiente al dominio asignado).
3. **Creación de base de datos:** desde cPanel, crear una base de datos MySQL nueva, un usuario con todos los privilegios sobre ella, e importar el esquema mediante phpMyAdmin (incluido en cPanel) ejecutando el script de creación de tablas.
4. **Configuración del .env:** crear el archivo `.env` en el servidor con las credenciales reales de producción: dominio en `APP_URL`, credenciales de la base de datos creada en el paso anterior, credenciales del servidor SMTP institucional de Casa Monarca, y un `JWT_SECRET` nuevo y aleatorio.
5. **Generación del certificado del sistema:** ejecutar el script de `setup/generar_cert_sistema.php` desde una terminal SSH si el plan de hosting lo permite, o mediante una invocación puntual vía navegador protegida temporalmente.
6. **Inicialización:** ejecutar `seed.php` una única vez visitando su URL desde el navegador, y guardar inmediatamente las credenciales del administrador generadas.
7. **Configuración de tareas cron:** desde la sección de Cron Jobs de cPanel, programar la ejecución diaria de `cron/avisos_expiracion.php` y `cron/verificar_hashes_adjuntos.php` mediante el binario de PHP del hosting.
8. **Verificación de permisos:** confirmar que el directorio de certificados tiene permisos restringidos (700 para el directorio, 600 para la llave privada del sistema, 644 para el certificado público), y que pertenece al usuario bajo el cual corre PHP en el hosting.

13.2. Entornos

Tabla 12: Comparación de entornos soportados por el sistema.

Aspecto	Desarrollo (Docker)	Producción (HostGator)
Servidor web	Apache en contenedor	Apache del proveedor
Base de datos	MySQL en contenedor	MySQL gestionado por cPanel
SMTP	Mailtrap (sandbox)	Servidor institucional real
APP_URL	<code>http://localhost:8080</code>	Dominio real de Casa Monarca
Acceso a logs	<code>docker compose logs</code>	Panel de logs de cPanel
Cron	Manual o crontab del host	Cron Jobs de cPanel

13.3. Consideraciones

- El archivo `.env` de producción **nunca** debe coincidir con el de desarrollo ni subirse al repositorio. Debe crearse directamente en el servidor de producción.
- El `JWT_SECRET` de producción debe ser distinto al de desarrollo y generado aleatoriamente con al menos 64 caracteres, ya que cualquier persona con acceso a este secreto podría falsificar tokens de sesión válidos.
- Los certificados del sistema generados en desarrollo **no** deben reutilizarse en producción. El script de generación debe ejecutarse de nuevo en el servidor de producción para que la CA institucional real sea distinta a la de pruebas.
- HostGator Plan Negocios tiene límites de recursos (CPU, memoria, conexiones concurrentes a base de datos) propios de hosting compartido. El volumen esperado de uso de Casa Monarca está dentro de estos límites, pero un crecimiento significativo de la organización podría requerir migración a un plan superior o a un VPS.
- Es recomendable realizar un respaldo completo de la base de datos antes de cualquier actualización del código en producción, dado que algunas migraciones de esquema no son reversibles automáticamente.

14. Seguridad

14.1. Autenticación

El sistema utiliza un esquema de autenticación basado en dos factores complementarios: contraseña hasheada con bcrypt para el login inicial, y certificado digital X.509 para operaciones que requieren firma. La sesión autenticada se mantiene mediante tokens JWT firmados con HS256, que incluyen en su payload el identificador del usuario, su nivel jerárquico y un timestamp de expiración.

```
$payload = [  
    'sub'    => $usuario['id'],  
    'nivel'  => $usuario['nivel'],  
    'area'   => $usuario['area'],  
    'exp'    => time() + 3600  
];  
$token = JWT::encode($payload, env('JWT_SECRET'), 'HS256');
```

Cada petición autenticada incluye este token en el header **Authorization**, y el middleware RBAC lo valida antes de permitir cualquier operación. Tokens expirados o con firma inválida son rechazados con código HTTP 401, forzando al frontend a redirigir al login.

14.2. Manejo de datos sensibles

14.2.1. Contraseñas

Las contraseñas nunca se almacenan en texto plano. Se hashean con bcrypt mediante la función nativa `password_hash()` de PHP, que aplica salt aleatorio automáticamente. La verificación se realiza con `password_verify()`, que compara el hash sin necesidad de descifrar nada.

14.2.2. Llaves privadas

Las llaves privadas de los usuarios siguen el principio de zero-knowledge: se generan en el servidor en el momento de emisión del certificado, se entregan al usuario mediante un link de descarga con expiración de 24 horas, y a partir de ese momento el servidor no las conserva. Todas las operaciones de firma posteriores ocurren localmente en el navegador del usuario mediante la Web Crypto API.

La única llave privada que reside permanentemente en el servidor es la del certificado dedicado del sistema, utilizada exclusivamente para firmar correos salientes con S/MIME.

Esta llave se almacena con permisos restrictivos (600) y pertenece únicamente al usuario `www-data` que ejecuta el proceso de Apache.

14.2.3. Contenido de documentos y comunicaciones

El contenido binario de documentos y adjuntos se almacena codificado en base64 en la base de datos. Es importante para cualquier desarrollador entender que **esto es codificación, no cifrado**: cualquiera con acceso directo a la base de datos puede decodificar el base64 y recuperar el contenido original. La integridad (no la confidencialidad) está garantizada mediante hashes SHA-256 y firmas digitales asociadas. El cifrado en reposo a nivel motor (TDE) queda identificado como mejora pendiente para la versión 2.

14.3. Buenas prácticas aplicadas

- **Prepared statements en toda interacción con la base de datos.** Ninguna consulta SQL del sistema concatena valores directamente; todas usan placeholders parametrizados a través de PDO, eliminando el vector de inyección SQL.
- **Validación de entrada en cada endpoint.** Los controladores validan la presencia y formato de los campos requeridos antes de procesar cualquier operación, devolviendo errores 400 descriptivos ante datos faltantes o inválidos.
- **Principio de mínimo privilegio en RBAC.** Cada operación requiere explícitamente el nivel y permiso mínimo necesario; no existen rutas que omitan la verificación de autorización.
- **Defensa en profundidad en validación de archivos.** Tres capas independientes de validación (extensión, MIME, magic bytes) más detección de macros en Office, en lugar de confiar en una sola verificación superficial.
- **No exposición de información sensible en mensajes de error.** Los mensajes de error son descriptivos para el usuario pero no revelan detalles internos de implementación que pudieran facilitar un ataque (por ejemplo, no se distingue entre “usuario no existe” y “contraseña incorrecta” en el login).
- **Separación de secretos en variables de entorno.** Credenciales de base de datos, secreto JWT y credenciales SMTP nunca están hardcodeadas en el código fuente, viven exclusivamente en `.env`, archivo excluido del control de versiones.

- **Permisos restrictivos de filesystem en material criptográfico.** Las llaves privadas almacenadas temporalmente o de forma permanente en el servidor tienen permisos 600, legibles únicamente por el usuario que ejecuta el proceso de PHP.
- **Bitácora inmutable como segunda línea de defensa.** Incluso si un control de acceso fallara, la bitácora permite reconstruir qué ocurrió y cuándo, facilitando respuesta a incidentes.

14.4. Recomendaciones para desarrolladores futuros

- Nunca modificar la lógica de `verificarAcceso` sin entender el impacto sobre todos los controladores que dependen de ella.
- Nunca registrar contenido de llaves privadas en logs, ni siquiera para depuración temporal.
- Al agregar un nuevo endpoint que reciba archivos, reutilizar siempre `ValidacionArchivos::validar()` en lugar de implementar validación ad-hoc.
- Al modificar el esquema de base de datos, considerar el impacto sobre la bitácora y asegurar que las nuevas operaciones sensibles también queden registradas.
- Rotar el `JWT_SECRET` periódicamente en producción invalida todas las sesiones activas; coordinar este cambio con los usuarios del sistema.

15. Guía de Contribución

15.1. Convenciones de código

15.1.1. PHP

- Los controladores siguen el patrón `NombreController.php` con métodos públicos que corresponden directamente a acciones de la API (`crear`, `listar`, `ver`, `actualizar`, etc.).
- Todo método de controlador que requiera autenticación debe invocar `verificarAcceso()` como primera línea de ejecución.
- Los identificadores de entidades se generan como UUID v4 mediante el método privado `uuid()` presente en cada controlador.

- Las consultas SQL siempre usan prepared statements con `?` como placeholder; no se permite concatenación de valores en las consultas.
- Los nombres de tablas y columnas en base de datos están en español y `snake_case` (ej. `fecha_expiracion`, no `expirationDate`).
- Los mensajes de error devueltos al cliente están en español, dado que el público objetivo del sistema es hispanohablante.

15.1.2. JavaScript

- Las funciones de render de página siguen el patrón `renderNombreSeccion()` (ej. `renderUsers`, `renderDocumentos`).
- Las funciones que abren modales siguen el patrón `abrirModalNombre()` u `openModalNombre()`.
- Toda llamada a la API pasa por el wrapper `API.get()` o `API.post()`, nunca se invoca `fetch` directamente salvo en casos que requieran `FormData` para archivos.
- Las variables de estado global relevantes (`SESSION`, `BASE_URL`) se declaran en mayúsculas al inicio del archivo.

15.2. Flujo de trabajo (Git)

1. Clonar el repositorio y levantar el sistema localmente siguiendo las instrucciones de la Sección 6.
2. Crear una rama nueva a partir de `main` con nombre descriptivo del cambio propuesto, siguiendo el patrón `tipo/descripcion-breve` (ej. `feature/notificaciones-in-app`, `fix/validacion-carpetas`).
3. Aplicar los cambios manteniendo las convenciones de código descritas anteriormente.
4. Antes de hacer commit, verificar que el archivo `.env` no esté incluido en los cambios mediante `git status`.
5. Realizar commits descriptivos siguiendo el formato `tipo: descripción` (ej. `feat: agregar notificaciones in-app`, `fix: corregir validación de carpeta sin clasificar`).
6. Hacer `pull -rebase` antes de cada push para mantener el historial lineal.
7. Abrir un pull request hacia `main` con explicación del cambio, justificación y pasos para reproducir las pruebas realizadas.

```
git checkout -b feature/nombre-descriptivo
# ... aplicar cambios ...
git status                # verificar que .env no aparece
git pull --rebase
git add .
git commit -m "feat: descripci n breve del cambio"
git push origin feature/nombre-descriptivo
```

15.3. Cómo agregar funcionalidades

Para agregar un módulo nuevo siguiendo la arquitectura existente, el flujo recomendado es:

1. **Definir el esquema de base de datos** necesario, con migraciones SQL que incluyan los UUID como llave primaria y las llaves foráneas apropiadas hacia **usuarios** si corresponde.
2. **Crear el controlador** en `controllers/` siguiendo el patrón de métodos públicos por acción, invocando `verificarAcceso()` en cada uno según el nivel y permiso requerido.
3. **Registrar las rutas** en el router de `api/index.php`, agregando el `case` correspondiente al nuevo recurso.
4. **Si la funcionalidad requiere firma digital**, reutilizar el patrón existente: la firma se genera en frontend con Web Crypto API sobre una cadena bien definida, y se verifica en backend con `openssl_verify` contra el certificado activo del usuario.
5. **Si la funcionalidad recibe archivos adjuntos**, reutilizar `ValidacionArchivos::validar()` antes de cualquier persistencia.
6. **Si la funcionalidad dispara correos**, agregar la plantilla correspondiente en `templates.php` y usar `Mailer::send()` o `Mailer::sendConAdjuntos()` según corresponda, garantizando que el correo salga firmado con S/MIME automáticamente.
7. **Agregar el registro en bitácora** dentro de la misma transacción que la operación principal, para mantener consistencia con el resto del sistema.
8. **Construir la interfaz frontend** siguiendo el patrón de funciones `render` y modales reutilizables, agregando la entrada correspondiente al sidebar si aplica según el nivel del usuario.

Cualquier cambio que afecte la lógica criptográfica, el modelo de control de acceso o la persistencia de información sensible requiere revisión adicional por parte de otro integrante del equipo antes de integrarse a `main`.

16. Problemas Conocidos

16.1. Bugs actuales

Al tratarse de la primera versión funcional completa del sistema, no se tienen bugs abiertos conocidos al momento de esta entrega. Todos los flujos críticos documentados en la tabla de casos de uso del reporte técnico fueron verificados y se comportan según lo esperado. Los problemas identificados durante el desarrollo fueron corregidos antes del cierre de cada sesión de trabajo, según el historial de commits del repositorio.

16.2. Limitaciones

16.2.1. Limitaciones del hosting compartido (HostGator Plan Negocios)

- **Sin acceso root ni configuración avanzada del servidor.** No es posible instalar extensiones de PHP adicionales a las que el proveedor ofrece por defecto, ni modificar configuraciones a nivel de sistema operativo. Cualquier extensión requerida (como `openssl` o `zip`) debe estar disponible en el plan contratado.
- **Recursos compartidos con otros clientes del proveedor.** El rendimiento puede verse afectado por la carga de otros sitios alojados en el mismo servidor físico, especialmente en picos de tráfico.
- **Sin soporte nativo para contenedores Docker en producción.** El despliegue en HostGator requiere ejecutar PHP directamente sobre el Apache del proveedor, sin la capa de aislamiento que ofrece Docker en desarrollo. Esto implica que las versiones exactas de PHP y MySQL en producción dependen de lo que el proveedor tenga disponible, debiendo verificarse compatibilidad antes del despliegue final.
- **Limitaciones en tareas cron.** El panel de cPanel permite programar tareas cron pero con granularidad y frecuencia mínima que pueden variar según el plan contratado; debe verificarse que el plan permita ejecuciones diarias sin restricciones adicionales.
- **Sin Transparent Data Encryption disponible.** La mayoría de los planes de hosting compartido no exponen control sobre el motor de almacenamiento de MySQL al nivel

necesario para configurar TDE, lo que refuerza por qué esta mejora queda pendiente para una infraestructura distinta en la versión 2.

- **Límite de tamaño de subida de archivos configurado por el proveedor.** Los valores de `upload_max_filesize` y `post_max_size` en `php.ini` pueden estar limitados por el plan de hosting, independientemente del límite de 10 MB que el sistema aplica a nivel de aplicación.

16.2.2. Limitaciones funcionales conscientes

- El sistema no realiza análisis de contenido (antivirus) sobre los archivos adjuntos, únicamente valida su formato.
- No existe autenticación de doble factor; la seguridad de la cuenta depende de la fortaleza de la contraseña y la posesión del certificado digital.
- Las carpetas personales por colaborador no están implementadas en esta versión.
- El módulo de reportes y estadísticas operativas no está implementado en esta versión.
- Los derechos ARCO conforme a la LFPDPPP no tienen un flujo formal dentro del sistema; su atención actualmente requeriría proceso manual fuera de la plataforma.

17. FAQ Técnica

17.1. Problemas comunes al levantar el proyecto

P: Los contenedores se levantan pero `http://localhost:8080` no responde.

R: Verificar que el puerto 8080 no esté ocupado por otro proceso en el equipo local con `lsof -i :8080` (macOS/Linux) o revisar el estado de los contenedores con `docker compose ps`. Si el contenedor `php` aparece reiniciándose constantemente, revisar sus logs con `docker compose logs php` para identificar el error de arranque.

P: Error de conexión a la base de datos al ejecutar `seed.php`.

R: Confirmar que el contenedor `mysql` terminó completamente su inicialización antes de ejecutar el seed; MySQL puede tardar entre 10 y 30 segundos en estar listo después de `docker compose up`. Verificar también que las credenciales en `.env` coincidan exactamente con las definidas en `docker-compose.yml`.

P: Los correos no llegan a Mailtrap durante desarrollo.

R: Verificar que `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME` y `MAIL_PASSWORD` en `.env` correspondan exactamente a las credenciales del inbox de Mailtrap creado para el proyecto.

Revisar también que no haya un firewall local bloqueando conexiones salientes al puerto 2525.

P: generar_cert_sistema.php falla con error de permisos.

R: El directorio `/var/certificados/sistema/` debe existir y ser escribible por el usuario que ejecuta PHP dentro del contenedor. Si el error persiste, ejecutar `docker compose exec php mkdir -p /var/certificados/sistema` y volver a intentar.

P: El frontend carga pero todas las peticiones a la API devuelven 404.

R: Confirmar que las rutas en el frontend usan `BASE_URL` correctamente definido como `window.location.origin`, y no una URL hardcodeada de una sesión de desarrollo anterior (especialmente relevante si se usó un tunnel temporal como ngrok o devtunnels en alguna sesión previa).

17.2. Errores típicos de desarrollo

P: “La firma digital no es válida” aunque el contenido parece correcto.

R: La causa más común es que la cadena firmada en el frontend no coincide exactamente, carácter por carácter, con la cadena reconstruida en el backend. Revisar especialmente espacios en blanco, saltos de línea y el orden de concatenación de los campos. Es recomendable hacer `console.log` de la cadena exacta firmada en frontend y compararla con un `error_log` de la cadena reconstruida en backend durante depuración.

P: Un archivo PDF válido es rechazado por la validación de adjuntos.

R: Verificar que los primeros bytes del archivo correspondan efectivamente al magic byte de PDF (25504446 en hexadecimal). Algunos editores de PDF o conversores generan archivos con cabeceras no estándar; en ese caso, abrir el archivo con un editor hexadecimal para confirmar su estructura real antes de reportarlo como bug.

P: Cambios en un controlador no se reflejan al probar la API.

R: Si se está trabajando dentro del contenedor Docker con volúmenes montados, confirmar que el archivo editado realmente está dentro del volumen sincronizado declarado en `docker-compose.yml`. En caso de duda, reiniciar el contenedor con `docker compose restart php`.

P: El cron de verificación de integridad no detecta una modificación de prueba.

R: Confirmar que la modificación se hizo directamente en la base de datos (por ejemplo, vía phpMyAdmin) y no a través de un endpoint de la aplicación, ya que cualquier flujo que pase por los controladores recalcula automáticamente el hash. El cron está diseñado para detectar manipulación *fuera* del flujo normal de la aplicación.

P: JWT inválido inmediatamente después de hacer login.

R: Verificar que `JWT_SECRET` en el archivo `.env` no haya cambiado entre la emisión del token (login) y la verificación (peticiones subsecuentes). Esto puede ocurrir si se reinició el contenedor con un `.env` modificado entre ambos pasos, invalidando todos los tokens emitidos previamente.

Anexo: Repositorio del proyecto

El código fuente completo del proyecto, junto con la documentación técnica, el manual de usuario, las evidencias de los casos de uso y los reportes ejecutivo y técnico, se encuentra disponible en el repositorio público de GitHub:

`https://github.com/carolavazquez/MA2006B`

El repositorio se distribuye bajo licencia MIT y contiene la estructura organizada en código fuente (`src/`), documentación (`docs/`), evidencias por caso de uso y el archivo `README.md` con las instrucciones de instalación, configuración y uso del sistema.

Referencias

Esta sección reúne enlaces de utilidad práctica para la instalación, configuración y uso de las dependencias del proyecto.

Docker y Docker Compose: `https://docs.docker.com/get-started/`

PHP 8.2 — Manual oficial: `https://www.php.net/manual/es/`

Extensión OpenSSL de PHP: `https://www.php.net/manual/es/book.openssl.php`

Extensión PDO de PHP: `https://www.php.net/manual/es/book.pdo.php`

MySQL 8 — Documentación oficial: `https://dev.mysql.com/doc/refman/8.0/en/`

Web Crypto API — MDN: `https://developer.mozilla.org/es/docs/Web/API/Web\_Crypto\_API`

Mailtrap — Servidor SMTP de pruebas: `https://mailtrap.io/`

phpMyAdmin: `https://www.phpmyadmin.net/docs/`

HostGator — Documentación de soporte para cPanel: `https://www.hostgator.mx/ayuda/`

MySQL Workbench (generación de diagramas ER): <https://dev.mysql.com/downloads/workbench/>

draw.io — Editor de diagramas: <https://app.diagrams.net/>